Log Anomaly Detection as a Trigger for Self-Healing in Kubernetes: An Empirical Study with LogBERT

Akella Harshith Saipaavan[1*], K. Gangadhara Rao[1], Ponna Manish Reddy[1], Bavikadi Mohaseen Hussain[1], M Jaidev[1]

[1*]Department of Information Technology, Chaitanya Bharathi Institute of Technology, Hyderabad, 500075, Telangana, India.

*Corresponding author(s). E-mail(s): harshithakella04@gmail.com;
Contributing authors: kgangadhar_it@cbit.ac.in;
manishreddypn@gmail.com; mohseensn7@gmail.com;
ugs24186_it.jaidev@cbit.org.in;

Abstract

Microservice applications on Kubernetes recover from faults only after users are affected. The platform restarts a container once its health check fails and adds capacity once CPU usage rises, so every fault costs failed requests before recovery begins. Container logs often show the first signs of trouble, and learned detectors such as LogBERT find anomalies in benchmark logs with high accuracy. These detectors, however, are mostly tested on stored datasets, and few studies check whether their alerts on live systems are fast and reliable enough to trigger recovery. We address this gap by connecting LogBERT to a controller that restarts or scales the affected service, and testing the loop on the Online Boutique application under injected pod kills, outages and CPU throttling. LogBERT detected all 18 test faults with a mean F1-score of 0.913 over five training runs, slightly above DeepLog (0.901) and a simple keyword rule (0.897). On the public HDFS logs it was the best unsupervised detector, but on BGL, where many later messages never appeared in training, every method failed. For sudden faults its alerts arrived together with the first user-visible error. Where both failed, the controller cut recovery time from 97.4 to 41.8 seconds compared with standard Kubernetes. Combined with the autoscaler, it raised availability from 92.1 to 93.3 percent, at 8.2 unnecessary actions per hour. Log-based recovery therefore shortens failures but cannot yet prevent them.

Keywords: AIOps, anomaly detection, fault injection, Kubernetes, log analysis, LogBERT, microservices, self-healing systems

1 Introduction

An application built from microservices is a set of small programs that call each other over the network [1]. Each service can be deployed and scaled on its own, which is why the style has spread, but the number of ways the whole system can fail grows with every service and every call between them. A cart service that loses its cache, a catalogue service that runs out of CPU or a single pod caught in a crash loop can stall the checkout path that users see.

Kubernetes [2] gives such applications basic self-repair. The kubelet restarts a container whose liveness probe stops answering, and the Horizontal Pod Autoscaler (HPA) adds replicas when the average CPU use of a Deployment crosses a target. Both mechanisms are reactive. A probe fails only after the container is already unresponsive, and an average over a metrics window moves only after the load has built up. Some faults are not covered at all: when a Deployment is scaled to zero by mistake, no probe runs and the HPA has nothing to measure.

Logs are a cheaper and often earlier signal. A healthy service writes a small, repetitive set of messages; a failing one writes timeouts, refused connections, retries and stack traces, and its callers start logging errors about it. Learned log anomaly detectors such as DeepLog [3] and LogBERT [4] model the order of these messages and report high accuracy on public benchmarks [5, 6]. Most of that work stops at the detection score, however, and evaluates on archived logs from systems such as HDFS or supercomputers. Whether the score is good enough to act on automatically, inside the orchestrator that runs the services, is a separate question that existing studies rarely test [7, 8].

This paper studies that question on a small, reproducible testbed. We compare a learned sequence model with simpler detectors on live microservice logs, identify the faults that leave too little trace in the logs and the parts of LogBERT that actually matter, and measure whether acting on log anomalies shortens recovery compared with the mechanisms Kubernetes already provides. Our contributions are as follows.

1. A closed-loop system that collects container logs through the Kubernetes API, parses them with Drain [9], scores per-service windows with LogBERT and maps persistent anomalies to restart or scale actions under a cool-down and an action budget (Section 3).
2. An evaluation protocol for live microservice logs: faults injected through the Kubernetes API, labels that follow the service call graph, an exclusion period after each fault, a separate validation run for every threshold and a prober that defines what a user-visible failure is (Section 4).
3. Evidence, over five training seeds, that on this application a keyword rule matches or beats LogBERT and DeepLog on faults that produce explicit errors while LogBERT is best on CPU throttling, and that LogBERT's additional components bring no

measurable gain; a check of the same detectors on the public HDFS and BGL logs under a chronological split; and a three-way closed-loop comparison showing that acting on the verdicts shortens recovery but raises availability only slightly, and only when combined with the autoscaler (Section 5).
4. Five practical lessons for log-driven remediation, drawn from what went wrong in the closed loop (Section 6).

In short, the detector found every injected fault, but a list of nine error words did nearly as well, and the log signal arrived together with the user-visible failure rather than before it. Acting on it cut recovery time where both setups failed, yet a plain autoscaler that happened to keep spare replicas prevented as many failures as the controller shortened; running the two together gave about one point more availability. On public benchmark logs the detector held its lead on one dataset and failed with all others on the second. We think these limits are worth reporting in detail, because most published log detectors are judged on archived benchmarks where neither of them can show up.

2 Related Work

2.1 Log-Based Anomaly Detection

Early detectors counted log events per window and applied PCA, invariant mining or clustering; He *et al.* [5] compared these methods on HDFS and BGL. DeepLog [3] replaced counting with an LSTM that predicts the next log key and flags a window when the observed key is not among the g most likely ones. LogAnomaly [10] added template semantics so that new but similar messages are not treated as anomalies, and LogRobust [11] used attention over semantic vectors to cope with logs that change between releases. LogBERT [4] applies a bidirectional Transformer encoder [12, 13] trained to predict masked log keys, together with a hypersphere objective adapted from Deep SVDD [14], and reported better F1-scores than DeepLog and LogAnomaly on HDFS, BGL and Thunderbird. NeuralLog [15] skips parsing and feeds raw messages to a Transformer. Le and Zhang [16] later showed that reported results depend strongly on how the data are split, on label noise and on parsing quality, which is one reason we keep a strict separation between validation and test runs.

2.2 Log Parsing

All key-based detectors depend on a parser that turns free-text messages into templates. Drain [9] builds a fixed-depth tree over message tokens, runs online and was among the most accurate parsers in the benchmark of Zhu *et al.* [17]. The Loghub collection [6] provides the datasets on which most parsers and detectors are compared.

2.3 Large Language Models for Log Analysis

Since 2023 large language models have been applied to both parsing and detection. LogPPT [18] parses logs by few-shot prompting of a pretrained model, and LILAC [19]

adds a parsing cache so that repeated messages do not trigger new queries. LogGPT [20], from the group that proposed LogBERT, trains a GPT-style model to predict the next log key and fine-tunes it with reinforcement learning. These models are far larger and slower per window than the 3.2 M-parameter encoder used here. We keep a small model because the detector has to score every window of every service in real time on a laptop GPU, and leave a comparison with LLM-based detectors to future work. Work since then has pushed in two directions. LogMoE [21] combines small expert models trained on several systems so that detection transfers to a new system without parsing, and LogPipe [22] matches log sequences against a growing knowledge base of patterns to reduce the number of LLM queries, reporting an average F1-score of 97.5% across eight public datasets. Both are evaluated on archived logs, and neither feeds its verdicts back into the running system.

2.4 Failure Management for Microservices

Surveys of AIOps [7] and of anomaly detection and root-cause analysis for microservices [8] show that most methods work on metrics or traces, and that remediation is studied far less than detection. Root-cause methods such as MicroRCA [23] rank suspect services from metric anomalies on the call graph. Benchmark applications such as DeathStarBench [24] and TrainTicket [25] and the Online Boutique demo [26] make such studies repeatable, and chaos engineering [27] supplies the fault injection. Our controller follows the monitor–analyse–plan–execute loop of autonomic computing [28], with logs as the monitored signal and the Kubernetes API as the effector.

Closer to our goal, Seer [29] trains a deep network on distributed traces to predict quality-of-service violations in microservices before they happen and signals the cluster manager to adjust resources, and FIRM [30] uses reinforcement learning to reprovision the microservices responsible for service-level-objective violations. Both act on traces and performance counters rather than logs. At the level of whole incidents, Ahmed *et al.* [31] use large language models to recommend root causes and mitigation steps, and RCACopilot [32] collects diagnostic data and predicts the root-cause category with an LLM; in both, engineers still carry out the mitigation. More recent systems let LLM agents act directly. KubeLLM [33] uses several cooperating agents to diagnose and repair misconfigured Kubernetes deployments, and Cornacchia *et al.* [34] found that tool-equipped agents improve root-cause analysis in microservices but remain overconfident and sometimes take wasteful or harmful actions. That last finding matches our own experience with automatic actions in Section 5.

3 System Architecture

Fig. 1 shows the system. All components except the services themselves run as processes on the host and talk to the cluster only through the Kubernetes API, so the application does not have to be changed.

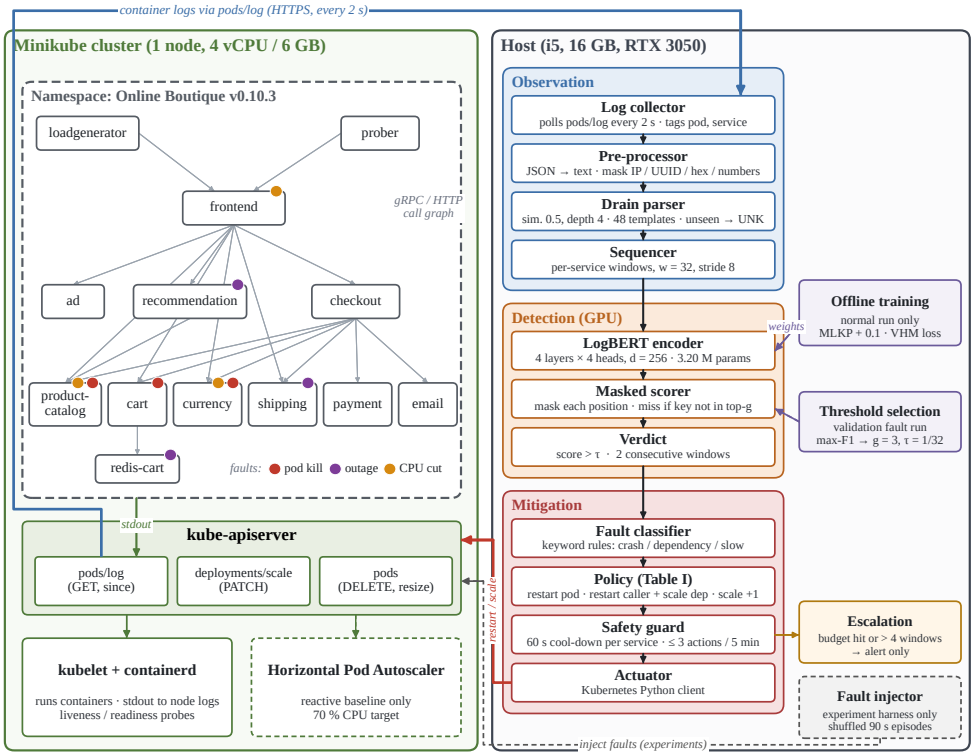


Fig. 1 System architecture. Left: the single-node Minikube cluster running Online Boutique; grey arrows are service calls and coloured dots mark the services targeted by each fault type. Right: the host-side pipeline. Blue: log flow from the API server to the collector. Red: mitigation actions. Dashed: paths used only during training and experiments.

3.1 Log Collection and Parsing

The collector polls the `pods/log` endpoint of every monitored pod every 2 s, asks only for lines newer than the last poll and tags each line with its pod and service. Online Boutique services write JSON; the pre-processor keeps the severity, the message and the class of any HTTP status code and drops the rest. Numbers, IP addresses, UUIDs and hexadecimal identifiers are then replaced by placeholders, and the service name is prefixed to the line so that the same message from two services yields two different keys.

Drain runs with a similarity threshold of 0.5 and a tree depth of 4. Templates are learned from the normal run only, which gives 48 templates. A message that matches no template at run time is mapped to a reserved unknown key instead of creating a new template, so the vocabulary of the detector stays fixed. The sequencer keeps a buffer per service and emits a window of $w = 32$ consecutive keys every 8 keys.

3.2 LogBERT Detector

Fig. 2 shows the detector. Each key in a window is embedded and added to a positional embedding and to an embedding of the service that produced the window. A Transformer encoder with 4 layers, 4 attention heads and a hidden size of 256 (3.20 M parameters) processes the window in both directions.

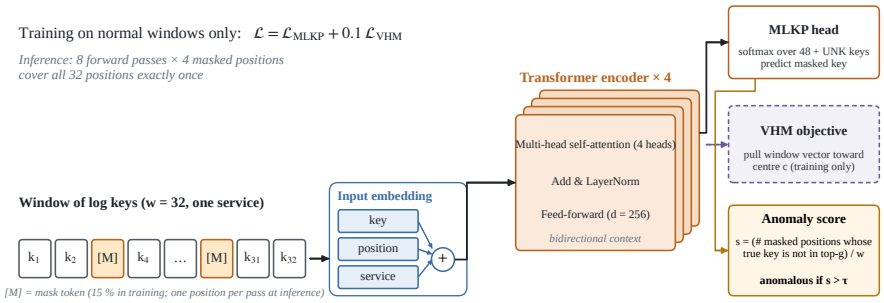


Fig. 2 LogBERT detector. Keys, positions and the service identity are embedded and summed, encoded by four Transformer layers and passed to a masked-key prediction head. The hypersphere (VHM) objective is used only in training. At inference every position is masked once and the anomaly score is the share of positions whose true key is not among the top-g predictions.

Training uses normal windows only and combines two self-supervised losses. In masked log-key prediction (MLKP), 15% of the positions are replaced by a mask token and the model predicts the original key from the context on both sides. The volume-of-hypersphere loss (VHM) pulls the pooled representation of each normal window towards a fixed centre $\mathbf{c}$, as in Deep SVDD [14]. The total loss is

$$\mathcal{L} = \mathcal{L}_{\text{MLKP}} + 0.1\,\mathcal{L}_{\text{VHM}}. \tag{1}$$

We train with AdamW (learning rate 10^{-3}, batch size 256, seed 0) for 40 epochs. The held-out loss stops improving after about 20 epochs while the training loss keeps falling, a sign of mild overfitting; we did not use early stopping.

At inference each of the 32 positions is masked exactly once, in 8 forward passes of 4 masked positions. A position is a miss when its true key is not among the g most probable keys, and the anomaly score of a window is

$$s = \frac{\text{number of misses}}{w}. \tag{2}$$

A window is anomalous when $s > \tau$. Both g and τ are chosen on the validation run by maximising the F1-score, which gives $g = 3$ and $\tau = 1/32 \approx 0.031$. The comparison is strict and τ equals the score of a window with exactly one miss, so a window is flagged only when at least two of its 32 keys are missed. Under normal load the log stream of each service is regular enough that the model rarely misses twice in one window.

3.3 Mitigation Controller

The controller (Fig. 3) acts on a service only after two consecutive anomalous windows from that service, which filters isolated spikes. It then assigns a coarse fault class from keywords in the raw lines of the anomalous window. Timeouts, refused connections and RPC errors mark a dependency fault, and any other service named in those lines is taken as the failing dependency. Panics, exceptions and out-of-memory messages mark a crash. A window with neither is treated as a slow handler. Table 1 maps each class to the least disruptive action we could think of for it.

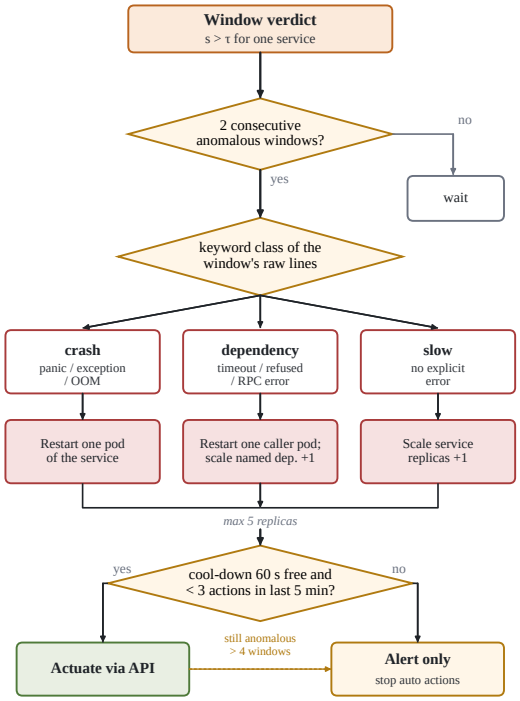


Fig. 3 Decision flow of the mitigation controller.

Two guards limit the damage a wrong verdict can do. Each service has a cool-down of 60 s between actions, and the controller as a whole may take at most three automatic actions in any five minutes; beyond that it only raises an alert. A restart deletes one pod at a time, so a Deployment with several replicas never loses all of them at once. The actuator uses the official Kubernetes Python client and needs only the permissions to delete pods and patch the scale of Deployments.

Table 1 Mitigation Policy

Suspected fault	Action
Crash or exception burst	Restart one pod of the service
Dependency error	Restart one caller pod; scale each named dependency by +1 (max. 5 replicas)
No explicit error (slow handler)	Scale the service by +1 (max. 5 replicas)
Still anomalous for $>$ 4 windows after an action	Alert only; stop automatic actions for the service

4 Experimental Setup

4.1 Testbed

All experiments run on one laptop (Intel Core i5-13450HX, 16 GB RAM, NVIDIA RTX 3050 Laptop GPU with 6 GB, Windows 11). Minikube 1.39 with the Docker driver hosts a single-node cluster running Kubernetes v1.37.0, limited to 4 vCPU and 6 GB of memory. The cluster runs release v0.10.3 of Online Boutique [26], which has 11 services written in Go, C#, Node.js, Python and Java plus a Redis cache; 10 services are monitored. Load comes from the bundled Locust load generator with its default 10 simulated users at a constant rate. A separate prober pod requests the home page, a product page and the cart in a loop with a 0.5 s pause and a 3 s timeout; a probe succeeds on any 2xx or 3xx response. A failed probe is our definition of a user-visible failure. LogBERT is trained and run on the laptop GPU; Table 2 lists the settings.

Table 2 Implementation Settings

Setting	Value
Log polling interval	2 s
Drain similarity / depth	0.5 / 4
Templates (incl. service prefix)	48
Window size / stride	32 / 8 keys
Encoder layers / heads / hidden size	4 / 4 / 256
Parameters	3.20 M
Mask ratio in training	15%
VHM weight	0.1
Training epochs	40 (about 8 s each on the GPU)
Optimizer / learning rate / batch	AdamW / 10^{-3} / 256
Top-g / threshold τ	3 / 1/32
Persistence	2 consecutive windows
Cool-down / action budget	60 s per service / 3 per 5 min

4.2 Fault Injection

Faults are injected through the Kubernetes API rather than a chaos tool, which keeps the setup to a single cluster without extra operators. Table 3 lists the three types and

their targets. Pod-kill episodes last 100 s and the others 90 s, the target and fault type follow a shuffled schedule, and episodes are separated by 120 s of normal traffic.

Table 3 Injected Fault Types

Fault	Mechanism	Targets
Pod kill	Force-delete the running pod every 20 s	productcatalog, currency, cart
Outage	Scale the Deployment to zero replicas	redis-cart, recommendation, shipping
CPU throttling	Cut the CPU limit of the running pod in place to 10 millicores	frontend, productcatalog, currency

4.3 Dataset and Labelling

Fig. 4 shows the protocol. The normal run contains no faults and is used to learn the Drain templates and to train every detector. It is not error-free: the load generator occasionally triggers a failed checkout or ad lookup, and those lines are kept so that the detectors learn them as normal. Two fault runs of 18 episodes each follow, one for validation and one for test, with different schedules. Each run covers every fault–target pair twice.

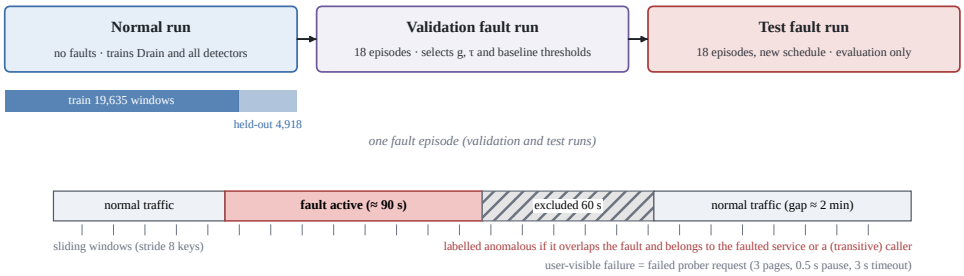


Fig. 4 Experimental protocol. The normal run trains all detectors; the validation run fixes every threshold; the test run is used only for evaluation. Bottom: labelling within one fault episode.

A window is labelled anomalous if it overlaps an injected interval and comes from the faulted service or from a service that calls it, directly or through other services, according to the static call graph of Online Boutique. Windows that start within 60 s after a fault ends are excluded from scoring, because the application is still recovering and neither label is clearly right. The normal run lasts 44.6 min and is split by time: windows from the first 32 min train LogBERT, windows from the last 12.5 min are held out, and the 37 windows that straddle the cut are dropped. The 196,999 normal lines in Table 4 are those of the 10 monitored services; the load generator and the

prober wrote another 35,458 lines that no detector sees. Consecutive windows share 24 of their 32 keys, so window-level counts are not independent samples; we therefore also report detection per fault episode (Section 5.2). Table 4 summarises the data.

Table 4 Dataset Summary

Item	Value
Monitored services / templates	10 / 48
Normal log lines	196,999
Training / held-out windows (normal run)	19,635 / 4,918
Validation windows (normal / anomalous)	21,557 / 6,640
Test windows (normal / anomalous)	21,900 / 6,443
Fault episodes (validation / test)	18 / 18

4.4 Baselines and Metrics

We compare LogBERT with three detectors. The keyword rule flags a window when any of its lines contains *error*, *fail*, *exception*, *fatal*, *panic*, *timeout*, *timed out*, *refused* or *unavailable*, ignoring case. Isolation Forest [35] uses 100 trees on template-count vectors. DeepLog [3] is a two-layer LSTM (embedding size 64, hidden size 128) that predicts each key from the previous 10, trained with Adam (learning rate 10^{-3}, batch size 1024) for 5 epochs; its validation-selected settings are $g = 2$ and a window threshold of 0.091. Both learned baselines are fitted on all 24,590 normal windows, including the part held out from LogBERT, which slightly favours them. Every threshold is chosen on the validation run and frozen for the test run. LogBERT and DeepLog are each trained with five seeds (0–4), and g and the threshold are chosen again on the validation run for every seed. We chose these baselines because each can score every window of every service in real time on the same laptop, which a closed loop requires; recent LLM-based detectors such as LogPipe [22] need model queries per sequence and are not designed for that setting. Public benchmarks such as HDFS and BGL [6] are recorded logs with no running system attached, so they can measure detection but not closed-loop recovery, which is the question this study asks. We therefore built the labelled testbed described above, and in addition ran the same detectors on HDFS and BGL to relate the results to public data (Section 5.4). During development the evaluation script read the test run three times, always with the same models and thresholds and with identical results, and later analyses (the seed repeats and the episode replay) used the same frozen thresholds; nothing was tuned on it.

The mitigation baseline is plain Kubernetes: the liveness and readiness probes shipped with Online Boutique, plus an HPA with a 70% CPU target on the services that receive the CPU-throttling fault. For detection we report window-level precision, recall and F1-score. For mitigation we report the detection lead time (from the first verdict on an affected service to the first failed probe, positive when the verdict comes first), the mean time to recovery (MTTR, from the first failed probe to the start of the first 10 s without a failed probe) and availability (the share of successful probes). Each closed-loop condition is measured in a single run.

The recovery comparison uses pod kill and CPU throttling only. A Deployment scaled to zero cannot be recovered by probes or the HPA until the fault is withdrawn, whereas the controller's scale action reverses it directly; including the outage fault in the main comparison would favour the controller by construction, so we report it separately. Pod kill is not biased in the same way, but Kubernetes already recreates a deleted pod by itself. For this fault a restart of the same pod gains nothing, so the comparison mainly tests whether the controller's actions on callers and dependencies help or interfere with the built-in recovery.

4.5 Use of AI Tools

The authors used Claude (Anthropic), an AI assistant, to help write code, run and monitor the experiment scripts on the authors' machine, prepare figures, and draft and edit the text of this manuscript. The authors designed the study, checked the code, verified every reported value against the experiment logs and result files, reviewed the full text and take responsibility for its content.

5 Results

5.1 Detection Quality

Table 5 gives the test results. For LogBERT and DeepLog it reports the mean and standard deviation over five training seeds; the keyword rule needs no training and Isolation Forest was run once. LogBERT has the highest mean F1-score, 0.913 ± 0.004, against 0.901 ± 0.005 for DeepLog and 0.897 for the keyword rule. LogBERT was ahead of DeepLog in every seed, by 0.005 to 0.024, so the gap is consistent but small. LogBERT's selected settings ($g = 3$, $\tau = 1/32$) were the same for every seed, while DeepLog's g ranged from 2 to 6. Seed 0 reproduced the original LogBERT model exactly (F1-score 0.912), and the per-fault, threshold, episode and ablation results below use that model. The methods differ more in how they err: the keyword rule has a precision of 0.997 but misses 18.5% of the anomalous windows, while LogBERT gives up about 2 points of precision for 4.3 points of recall. Isolation Forest on template counts is far behind (0.419), with most of its alarms false. Count vectors discard the order of messages, and a likely reason is that ordinary changes in traffic volume move them about as much as a fault does.

Table 5 Detection Results on the Test Run (Mean ± Standard Deviation over Five Seeds for the Learned Sequence Models)

Method	Precision	Recall	F1
Keyword rule	**0.997**	0.815	0.897
Isolation Forest[a]	0.324	0.591	0.419
DeepLog	0.962 ± 0.018	0.847 ± 0.017	0.901 ± 0.005
LogBERT	0.976 ± 0.005	$\mathbf{0.858 \pm 0.006}$	$\mathbf{0.913 \pm 0.004}$

[a]Single run.

Fig. 5 splits the F1-score by fault type. Pod kills and outages are loud: the affected service stops logging and its callers start logging refused connections and RPC errors, so the keyword rule does best on them (0.938 and 0.969), with LogBERT close behind (0.930 and 0.934) and DeepLog at 0.905 and 0.885. CPU throttling is quiet. A throttled service keeps writing the same messages, only more slowly, and its callers log errors only when their requests time out. Here the ranking reverses: LogBERT reaches 0.612, DeepLog 0.577 and the keyword rule only 0.487, so the learned models' advantage lies exactly on the fault that produces few error messages. Over the five seeds, LogBERT reaches 0.935 ± 0.004 on pod kill, 0.943 ± 0.008 on outage and 0.607 ± 0.028 on CPU throttling, and DeepLog 0.920 ± 0.014, 0.915 ± 0.029 and 0.558 ± 0.082, so LogBERT stays ahead of DeepLog on every fault type, while the keyword rule still leads on outages. Still, a model that sees only the order of keys cannot see that they have slowed down; adding inter-arrival times to the input is the most direct fix.

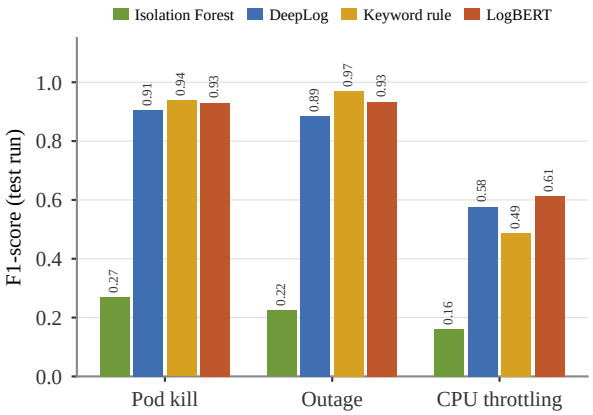


Fig. 5 F1-score per fault type on the test run (seed-0 models), computed on the windows of that fault together with all normal windows. The keyword rule leads on the loud faults, LogBERT on CPU throttling.

Fig. 6 shows how the LogBERT result on the test run moves with the threshold. Precision is already above 0.99 at $\tau = 0.1$ and stays there, while recall falls steadily, so every threshold above the validation choice gives up recall for almost no gain in precision. The F1-score on the test run peaks near $\tau = 0.05$, close to the value selected on the validation run. The test F1-score at the chosen threshold, 0.91, is within about 0.01 of the best point on the curve, so fixing τ on a separate run cost almost nothing here.

5.2 Episode-Level Detection

Window-level scores give long episodes more weight and count overlapping windows separately. Table 6 therefore reports, for each fault type, how many of the six test episodes produced at least one verdict on an affected service and how long after injection LogBERT's first verdict came. The numbers come from replaying the stored

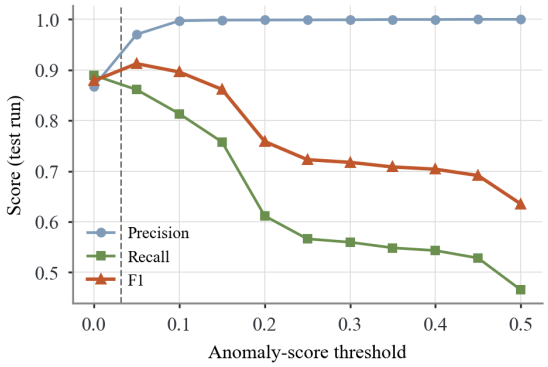


Fig. 6 Precision, recall and F1-score of LogBERT on the test run as a function of the anomaly-score threshold. The dashed line marks the value chosen on the validation run, $\tau = 1/32$.

window scores offline, not from the live controller. DeepLog's trained model was not kept, so it is omitted here.

Table 6 Episode-Level Detection on the Test Run (Offline Replay)

	Episodes detected (of 6)		LogBERT median time
Fault	LogBERT	Keyword rule	to first verdict (s)
Pod kill	6	6	2.3
Outage	6	6	3.9
CPU throttling	6	5	4.5
Total	18	17	–

LogBERT detects all 18 episodes and the keyword rule 17, but the counts flatter both methods. For pod kill and outage, LogBERT's first verdict arrives at the same moment as the first failed probe, so it is a detection, not a warning. Only CPU throttling gives a lead, of 2.4 to 23.0 s in four of its six episodes. The detector is also not silent between faults: it produced 89 false verdict windows on the normal windows of the 1.13 h test run. Against that background, a verdict somewhere inside a 90 s episode can be luck, and we suspect it was in the two throttling episodes where no probe failed at all.

5.3 Ablation

Table 7 changes one component at a time. None of the three changes to the model lowers the F1-score; removing the VHM objective, removing the service embedding or halving the encoder gives an F1-score of 0.916–0.917, with higher precision and slightly lower recall. With only 48 templates and highly regular normal traffic, masked-key prediction by a small encoder already captures what there is to learn, and the additional components bring no measurable benefit on this application. The last row

applies the controller's two-window persistence rule to the full model's window verdicts: precision rises to 0.984 and recall falls to 0.843 (F1-score 0.908), which is the trade the controller accepts to avoid acting on single windows.

Table 7 Ablation on the Test Run

Variant	Precision	Recall	F1
Full model	0.970	0.861	0.912
Without VHM objective	0.986	0.858	0.917
Without service embedding	0.983	0.858	0.916
Encoder with 2 layers	0.985	0.858	0.917
Full model + 2-window persistence	0.984	0.843	0.908

5.4 Public Benchmarks

To relate the testbed results to published work, we ran the same four detectors on the HDFS and BGL datasets of Loghub [6]. Both use a chronological split: the first 60% of the data trains the models on normal sequences only, the next 20% selects g and every threshold, and the last 20% is evaluated once. For HDFS we use Loghub's preprocessed block sessions (575,061 sessions, 16,838 of them anomalous, 29 templates), ordered by the first line of each block; LogBERT scores a session by its highest window score, and the keyword rule reads the template text. For BGL we fit Drain with the testbed settings on the first 60% of the 4,747,936 lines only, which gives 324 templates, and cut the log into 148,373 non-overlapping windows of 32 lines, 13,128 of which contain an alert. LogBERT uses the testbed architecture without the service embedding and is trained for 10 epochs instead of 40 to fit the time available. Table 8 gives the results.

Table 8 Detection Results on the Test Period of HDFS and BGL (Chronological 60/20/20 Split)

	HDFS			BGL		
Method	P	R	F1	P	R	F1
Keyword rule	1.000	0.489	0.657	0.170	0.599	**0.265**
Isolation Forest	0.022	0.917	0.043	0.041	0.177	0.067
DeepLog	0.685	0.645	0.664	0.122	0.827	0.213
LogBERT	0.864	0.721	**0.786**	0.083	0.985	0.152
LogRobust-style[a]	0.969	0.996	0.983	–	–	–

[a]Supervised: trained on labelled anomalies, which the other detectors never see. HDFS only.

On HDFS, LogBERT is the best of the unsupervised detectors, 0.12 ahead of DeepLog. Every method scores lower on the test period than on validation (LogBERT 0.933 on validation), because normal sessions near the end of the log are cut off by the end of collection: their median length is 13 events against 19 in training, and they contain far fewer delete events. The sequence models read such a session as a normal

prefix, whereas Isolation Forest's count vectors treat it as anomalous, and its F1-score falls to 0.043. On BGL every method fails and LogBERT is last. After the training cut, 28.5% of lines match no template learned before it, so the sequence models flag most windows (LogBERT has a recall of 0.985 at a precision of 0.083), and the keyword rule is misled by normal BGL messages that contain words such as *error*, for example "instruction cache parity error corrected".

These numbers are below most published results on both datasets. Le and Zhang [16] showed that random splits and parsing choices raise such results, and a chronological split with a parser frozen at the training cut is the stricter setting. We did not run a random-split protocol, so we cannot say how much of the gap the split explains. For reference, we also trained a supervised LogRobust-style detector [11] on HDFS: a two-layer bidirectional LSTM with attention pooling over frozen BERT vectors of each template, trained for 5 epochs with a class-weighted loss on all sessions of the first 60%, including the 11,158 anomalous ones. It reaches an F1-score of 0.983. It differs from the original in its event vectors and was run with one seed, and because it learns the anomaly types that recur through HDFS, it is a reference point rather than a like-for-like competitor. Taken together, the benchmark runs support the testbed results in one respect and limit them in another: LogBERT's lead over DeepLog holds on a second system, but a detector whose vocabulary is fixed at training time breaks down when the logging itself changes.

5.5 Early Warning and Recovery

We ran one new schedule of 18 episodes, used by neither the validation nor the test run, three times: with plain Kubernetes, with the controller alone and with the controller together with the same HPA as the baseline. In the reactive run the detector ran in observe-only mode, so its verdict times were recorded but no action was taken. In every run each Deployment was set back to one replica 60 s after each episode. Table 9 summarises pod kill and CPU throttling; we first compare the first two runs and then the third.

Table 9 Mitigation Outcomes on Pod Kill and CPU Throttling (12 Episodes, One Run Each)

Metric	Reactive	Controller	Controller + HPA
Episodes with failed probes (of 12)	4	10	4
MTTR, 4 episodes where reactive and controller failed (s)	97.4	41.8	–
MTTR, all episodes with failures (s)	97.4	32.2	53.6
Availability (%)	92.07	92.43	**93.33**
Failed probes during faults (%)	12.6	11.0	9.6
Automatic actions, whole run	–	36	37
Unnecessary actions per hour	–	4.5	8.2
Median detection lead time (s)	0.1[a]	0.0	4.8[b]

[a]Detector in observe-only mode. [b]One CPU-throttling episode with a 9.5 s lead; the pod kills have a median of 0.12 s.

Fig. 7 shows every episode of both runs. Only the paired MTTR row of Table 9 compares like with like; the row below it averages over different episodes, because the two runs failed in different places. In the four episodes in which both runs had failed probes, the controller recovered faster in every case, with a mean of 41.8 s against 97.4 s; the clearest cases are the two cart-service kills (86.5 and 28.0 s against 137.5 and 124.9 s). In six further episodes only the controller run had failed probes. In five of them the loss was small (2.3–4.5% of probes), and in one currency-service kill it was 40.7%. None of these failures was triggered by a controller action: in all six the first failed probe came before any frontend restart. In four of them the controller restarted the frontend 0.6–1.8 s after the failures began, in reaction to them, and each burst of failures ended within 11–19 s. In the three pod-kill episodes among those four, later kills of the same service caused no further failures once the controller had scaled it by one replica.

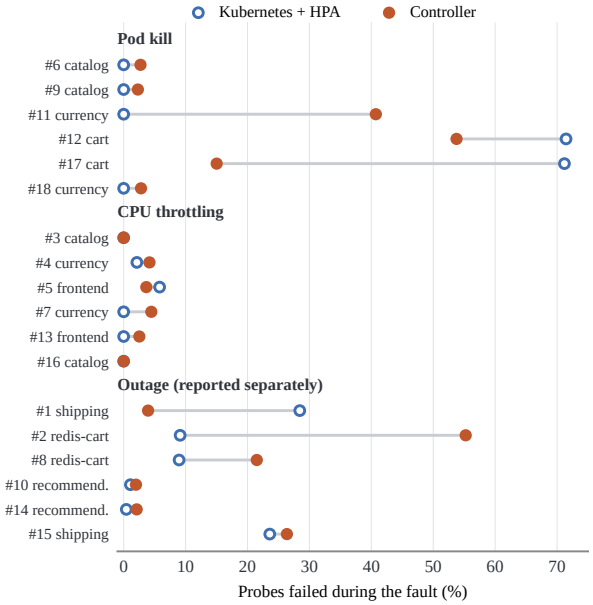


Fig. 7 Share of probes failed during each fault in the paired closed-loop runs (same schedule, episode number in front of the target). Where only one marker is visible the two runs coincide.

We attribute the gap to the baseline. After the frontend was throttled in the fifth episode of the reactive run, the HPA scaled the frontend to four or five replicas and kept them, overriding the reset, so later faults met a better-provisioned frontend than in the controller run, which had no HPA. This is consistent with the HPA's default five-minute scale-down stabilisation window, which outlasts the 60 s reset. The test run, which had neither, lost 26–73% of probes to the same pod kills and 31–41% to frontend throttling, against 2.3–4.5% in five of the six controller-run episodes. The net effect on availability is close to zero (92.43% against 92.07%). The controller shortens failures once they start; the HPA's lasting scale-up stops some of them from starting.

Over the whole run, outages included, availability was slightly lower with the controller (90.47% against 91.14%). The controller also took 4.5 unnecessary actions per hour.

The controller did not act early either. With the detector observing only, its median lead over the first failed probe was 0.1 s; with the controller acting it was 0.0 s. As in the offline replay, only CPU throttling gave a real lead, with a median of 19.1 s over the two throttling episodes of the reactive run that had failed probes.

To test the replica explanation, the third run gave the controller the same HPA as the baseline and logged the replica count of every Deployment at the start of each episode. The log confirms that the reset to one replica does not hold while an HPA is present: the frontend ran two to three replicas in the first four episodes and four from the fifth on, and the currency service ran four or five from the fifth episode on. The first episode already began with two frontend replicas, because the controller had scaled the frontend during warm-up, before any fault. With the same replica help as the baseline, the controller run had failed probes in four episodes, as many as the reactive run, and lost the fewest probes during faults (9.6% against 12.6%). In the three episodes in which both this run and the reactive run failed, it recovered in 28.9, 74.7 and 110.9 s against 45.1, 137.5 and 124.9 s. Availability over pod kill and CPU throttling was 93.33%, the highest of the three runs and 1.3 points above the reactive run. The price is more false actions, 8.2 per hour against 4.5 for the controller alone; a likely cause, which we have not verified, is that the extra replicas add log streams that raise false verdicts. Its median lead of 4.8 s comes from a single throttling episode and is not early warning for the abrupt faults. With one run per condition, a difference of about one point in availability is suggestive rather than established.

For the outage fault, reported separately for the reason given in Section 4, the MTTR is 73.4 s for the baseline, 56.1 s for the controller and 54.0 s for the controller with the HPA, with 11.9%, 18.5% and 16.9% of probes failing during the fault. The controller ended one shipping-service outage after 39.9 s instead of 125.6 s, but it made both Redis outages worse (55.2% and 21.5% of probes failed against 9.1% and 9.0%), because it restarted callers that could not recover while the cache was still down.

6 Discussion

6.1 Summary of Findings

Detection quality. A learned detector did not clearly outperform simple methods on the testbed. LogBERT has the highest window-level F1-score over five seeds, but DeepLog is only 0.012 and the keyword rule 0.016 behind it, and the rule is better on outages. On HDFS LogBERT leads the unsupervised detectors by a wider margin, while on BGL no method is usable under a chronological split. LogBERT's advantage is confined to CPU throttling, and across all faults it finds more anomalous windows than the rule at the cost of some false alarms. Per episode, LogBERT detects all 18 test faults and the rule 17.

Faults the logs barely show. CPU throttling is the hardest fault for every method, with LogBERT and DeepLog falling to 0.61 and 0.58. None of LogBERT's additions to a plain masked-key encoder, namely the hypersphere objective, the service embedding and the extra layers, improved the result on this 48-template vocabulary.

Closed-loop recovery. Acting on log anomalies helped only a little. When both the controller and plain Kubernetes let requests fail, the controller recovers faster (41.8 s against 97.4 s over four paired episodes), but on its own it left availability over pod kill and CPU throttling almost unchanged (92.43% against 92.07%). The failures that occurred only under the controller began before any controller action and are explained by the extra replicas that the HPA kept in the reactive run, which the replica log of the third run confirmed. With the same HPA on both sides, the controller gave the best availability of the three runs (93.33%) and recovered faster in all three episodes that failed in both, at the cost of 8.2 unnecessary actions per hour. For abrupt faults its verdicts arrive no earlier than the first failed request. With one run per condition, these differences remain open to revision.

6.2 Limits of Proactive Mitigation from Logs

The episode results limit what "proactive" can mean on this testbed. A pod kill or an outage breaks requests at once, and the logs report the failure as it happens, so a log-based controller cannot act before users are affected; it can only act sooner and on a better target than probes and autoscaling. Early warning appeared only for CPU throttling, where performance degrades gradually and LogBERT's first verdict preceded the first failed probe by up to 23 s in the offline replay and 38 s in the closed loop. Proactive mitigation from logs is therefore most promising for gradual faults such as resource exhaustion or slow dependencies, which are also the faults that sequence models currently detect worst.

6.3 Lessons for Log-Driven Remediation

The closed-loop runs exposed five problems that we expect any similar system to meet.

1. *Replica state decides the comparison.* The HPA's scale-down delay kept the baseline over-provisioned across episodes despite our reset, which changed the outcome more than any detector setting; once both sides had the HPA, the controller's advantage reappeared. Evaluations of remediation must record and control replica counts per episode.
2. *Scale before restarting.* Deleting the only pod of a service leaves it without a ready replica until the new pod starts; a rolling restart that starts the new pod before removing the old one, or a scale-up followed by a restart, avoids that window.
3. *Restarting callers does not repair a dead dependency.* During both Redis outages the controller restarted callers that could not recover while the cache was down, and more requests failed than without it. The policy needs to tell a failing service from a failing dependency before acting.
4. *Fault attribution from text is fragile.* Our first keyword classifier matched the `/cart` request path and named the cart service for almost every fault; reading only the error message fixed it. Rules over raw log text need checking against request fields that happen to contain service names.
5. *Logs give no warning before abrupt faults.* Every lead time for pod kills and outages was at most a few seconds, apart from one pod-kill verdict that came 55 s early and

was most likely a false alarm, so a log-driven controller can only react faster, not earlier, to such faults.

6.4 The Value of a Learned Detector

On Online Boutique, a keyword rule matches LogBERT on window-level F1 with almost no false alarms, and beats it on pod kills and outages. This says more about the faults than about the detectors: a killed pod or a missing dependency makes its callers log explicit errors, and nine words are enough to catch them. A learned detector needs no such word list, which matters for services whose messages nobody has read, and it reacts to changes in the order of ordinary messages, the only symptom a quiet fault may leave. The CPU-throttling results show that advantage, with LogBERT at 0.612 against the rule's 0.487, and also its limit: when the symptom is in timing rather than order, even the best model misses much of the fault. A practical deployment could run the cheap rule and the learned model side by side and act when either fires. The BGL results add a caveat: a learned detector whose vocabulary is fixed at training time fails once the system starts writing messages it has not seen, so it needs regular retraining or a parser that can absorb new message types.

6.5 Threats to Validity

The study uses one demo application, synthetic faults and a single-node cluster on a laptop. Production logs are noisier, change with every release [11] and arrive at much higher volume. Only three fault types are covered, so memory leaks and network faults are not evaluated. Detection results for LogBERT and DeepLog are averaged over five training seeds, but the fault runs themselves were recorded once, and each closed-loop condition is a single run, so the closed-loop differences carry no confidence intervals. The benchmark runs use one chronological split, Loghub's templates for HDFS and 10 training epochs for LogBERT, and the supervised baseline is a LogRobust-style reimplementation rather than a faithful reproduction. Because consecutive windows overlap by 75%, the 28,343 test windows contain far fewer independent observations than their number suggests; the 18 test episodes are the better guide to how uncertain the results are. Labels follow fault intervals and the static call graph, which blurs the exact onset and can mark a caller as anomalous even if it never noticed the fault. The fault class comes from keyword rules rather than a learned classifier, and the policy has no reschedule or resource-request action. The CPU-throttling fault lives in the running pod, so a restart removes it; a throttle caused by the workload itself would not go away that way. Logs are polled every 2 s, which bounds how early a verdict can arrive and can lose the last lines of a container that crashes between polls. In the first closed-loop comparison the HPA kept extra frontend replicas across episodes despite the reset, so the two runs did not start every episode from the same state; the third run addresses this, but its controller also scaled the frontend during warm-up. The controller's dependency names come from keyword rules that we refined on the validation run. LogBERT was trained for a fixed 40 epochs although its held-out loss stopped improving after about 20, and the learned baselines saw more normal data than LogBERT did. Finally, automatic remediation can do harm when the detector is

wrong, which is why the controller works under an action budget and hands persistent anomalies to a human.

7 Conclusion

We connected a LogBERT log anomaly detector to a guarded mitigation controller for Kubernetes and evaluated it on Online Boutique with three injected fault types. With thresholds fixed on a separate validation run, LogBERT reaches a mean F1-score of 0.913 over five seeds, slightly but consistently above DeepLog and level with a keyword rule. On the public HDFS logs it is the best unsupervised detector, while on BGL, whose messages change after the training cut, every method fails. It flags every test episode within a few seconds, but for pod kills and outages no earlier than the first user-visible failure, and all sequence models struggle with CPU throttling, whose symptoms are in timing rather than message order. LogBERT's hypersphere objective, service embedding and extra layers did not improve the result on this 48-template vocabulary. Acting on the verdicts shortened recovery when both the controller and plain Kubernetes let requests fail, but on its own did not raise availability, because the autoscaler's lasting scale-up in the baseline prevented failures that the controller could only shorten. With the autoscaler on both sides, the controller gave about one point more availability at almost twice the false-action rate. Next steps are repeated closed-loop runs with confidence intervals, a controller that scales up before it restarts a single-replica service, memory and network faults, inter-arrival times as an input feature, a parser that can absorb new message types, and a second application such as TrainTicket [25].

Declarations

- **Funding.** The authors did not receive support from any organization for the submitted work.
- **Conflict of interest.** On behalf of all authors, the corresponding author states that there is no conflict of interest.
- **Ethics approval and consent to participate.** Not applicable.
- **Consent for publication.** Not applicable.
- **Data availability.** The fault schedules (or the seeds that generate them), result files and scripts that regenerate the testbed logs are available at https://github.com/Harshith-Akella-1/logbert-k8s-mitigation. Raw log files larger than 50 MB are not included and can be regenerated with these scripts. The HDFS and BGL datasets are publicly available from Loghub [6].
- **Materials availability.** Not applicable.
- **Code availability.** The source code and Kubernetes manifests are available under the MIT license at https://github.com/Harshith-Akella-1/logbert-k8s-mitigation.
- **Author contributions.** A.H.S.: conceptualization, methodology, software, investigation, formal analysis, writing (original draft). K.G.R.: supervision, methodology, writing (review and editing). P.M.R.: software (fault injection and closed-loop experiments), validation, data curation. B.M.H.: software (log collection and parsing),

investigation, visualization. M.J.: literature review, validation, writing (review and editing). All authors read and approved the final manuscript.

References

[1] N. Dragoni, S. Giallorenzo, A. Lluch Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*. Cham, Switzerland: Springer, 2017, pp. 195–216.

[2] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *ACM Queue*, vol. 14, no. 1, 2016.

[3] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2017, pp. 1285–1298.

[4] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log anomaly detection via BERT," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2021, pp. 1–8.

[5] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis for anomaly detection," in *Proc. IEEE 27th Int. Symp. Softw. Rel. Eng. (ISSRE)*, 2016, pp. 207–218.

[6] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," in *Proc. IEEE 34th Int. Symp. Softw. Rel. Eng. (ISSRE)*, 2023.

[7] P. Notaro, J. Cardoso, and M. Gerndt, "A survey of AIOps methods for failure management," *ACM Trans. Intell. Syst. Technol.*, vol. 12, no. 6, Art. no. 81, 2021.

[8] J. Soldani and A. Brogi, "Anomaly detection and failure root cause analysis in (micro)service-based cloud applications: A survey," *ACM Comput. Surv.*, vol. 55, no. 3, Art. no. 59, 2022.

[9] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2017, pp. 33–40.

[10] W. Meng *et al.*, "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2019, pp. 4739–4745.

[11] X. Zhang *et al.*, "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, 2019, pp. 807–817.

[12] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017.

[13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics (NAACL-HLT)*, 2019, pp. 4171–4186.

[14] L. Ruff *et al.*, "Deep one-class classification," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 4393–4402.

[15] V.-H. Le and H. Zhang, "Log-based anomaly detection without log parsing," in *Proc. 36th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, 2021, pp. 492–504.

[16] V.-H. Le and H. Zhang, "Log-based anomaly detection with deep learning: How far are we?" in *Proc. 44th Int. Conf. Softw. Eng. (ICSE)*, 2022, pp. 1356–1367.

[17] J. Zhu *et al.*, "Tools and benchmarks for automated log parsing," in *Proc. 41st Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*, 2019, pp. 121–130.

[18] V.-H. Le and H. Zhang, "Log parsing with prompt-based few-shot learning," in *Proc. 45th Int. Conf. Softw. Eng. (ICSE)*, 2023.

[19] Z. Jiang *et al.*, "LILAC: Log parsing using LLMs with adaptive parsing cache," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, 2024.

[20] X. Han, S. Yuan, and M. Trabelsi, "LogGPT: Log anomaly detection via GPT," in *Proc. IEEE Int. Conf. Big Data*, 2023.

[21] J. Qi, Z. Luan, S. Huang, C. Fung, Y. Wang, A. Wang, H. Zhang, H. Yang, and D. Qian, "LogMoE: Lightweight expert mixture for cross-system log anomaly detection," in *Proc. 40th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, 2025.

[22] Y. Tao, H. Zhang, V.-H. Le, and Y. Xiao, "Knowledge-augmented log anomaly detection with large language models," in *Proc. 48th IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, 2026.

[23] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "MicroRCA: Root cause localization of performance issues in microservices," in *Proc. IEEE/IFIP Netw. Oper. Manage. Symp. (NOMS)*, 2020, pp. 1–9.

[24] Y. Gan *et al.*, "An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems," in *Proc. 24th Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2019, pp. 3–18.

[25] X. Zhou *et al.*, "Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study," *IEEE Trans. Softw. Eng.*, vol. 47, no. 2, pp. 243–260, 2021.

[26] Google Cloud Platform, "Online Boutique (microservices-demo)," GitHub repository, release v0.10.3. [Online]. Available: https://github.com/GoogleCloudPlatform/microservices-demo

[27] A. Basiri *et al.*, "Chaos engineering," *IEEE Softw.*, vol. 33, no. 3, pp. 35–41, 2016.

[28] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.

[29] Y. Gan *et al.*, "Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices," in *Proc. 24th Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2019, pp. 19–33.

[30] H. Qiu, S. S. Banerjee, S. Jha, Z. T. Kalbarczyk, and R. K. Iyer, "FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices," in *Proc. 14th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2020, pp. 805–825.

[31] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan, "Recommending root-cause and mitigation steps for cloud incidents using large language models," in *Proc. 45th Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 1737–1749.

[32] Y. Chen *et al.*, "Automatic root cause analysis via large language models for cloud incidents," in *Proc. 19th Eur. Conf. Comput. Syst. (EuroSys)*, 2024.

[33] M. De Jesus, P. Sylvester, W. Clifford, A. Perez, and P. Lama, "LLM-based multi-agent framework for troubleshooting distributed systems," in *Proc. IEEE Cloud Summit*, 2025.

[34] A. Cornacchia, I. Alabdulaal, I. Saghier, A. Mirdad, O. Fayoumi, and M. Canini, "Between promise and pain: The reality of automating failure analysis in microservices with LLMs," in *Proc. 16th ACM SIGOPS Asia-Pacific Workshop Syst. (APSys)*, 2025.

[35] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.